Low-Complexity Precision-Scalable Multiply-Accumulate Unit Architectures for Deep Neural Network Accelerators

Wenjie Li, Aokun Hu, Gang Wang, Ningyi Xu, and Guanghui He, *Member, IEEE*

***Abstract*—Precision-scalable deep neural network (DNN) accelerator designs have attracted much research interest. Since the computation of most DNNs is dominated by multiply-accumulate (MAC) operations, designing efficient precision-scalable MAC (PSMAC) units is of central importance. This brief proposes two low-complexity PSMAC unit architectures based on the well-known one, *Fusion Unit (FU)*, which is composed of a few basic units called *Bit Bricks (BBs)*. We first simplify the architecture of BB through optimizing some redundant logic. Then a top-level architecture for PSMAC unit is devised by recursively employing BBs. Accordingly, two low-complexity PSMAC unit architectures are presented for two different kinds of quantization schemes. Moreover, we provide an insight into the decomposed multiplications and further reduce the bitwidths of the two architectures. Experimental results show that our proposed architectures can save up to 44.18% area cost and 45.45% power consumption when compared with the state-of-the-art design.**

***Index Terms*—Deep neural networks (DNNs), multiply-and-accumulate (MAC), precision-scalable, low-complexity architecture.**

I. Introduction

DEEP neural networks (DNNs) have achieved a great success over the past few years. They not only beat the records in image classification and speech recognition, but also have been employed in a wide range of applications such as object detection and natural language processing. To deal with the tremendous amount of computation, hardware acceleration for DNNs has become an overwhelming trend [1]. For almost all DNNs, the computation is dominated by multiply-accumulate (MAC) operations. Therefore MAC units occupy an important place in DNN accelerator design.

In addition to reducing the model size, quantization is essential for DNN hardware implementations. It maps float-point weights or/and activations to a smaller set of fixed-point values. With the assistance of quantization, one can implement a DNN accelerator using only fixed-point arithmetic units. When adopting advanced quantization techniques, the bitwidth varies significantly across DNNs and even across layers within a DNN [2], [3], [4]. To maximize the utilization of MAC units under different precision configurations, precision scalability is supported by many DNN accelerators. MAC units of these precision-scalable accelerators are built either on decomposed multipliers [8], [10], [11] or by bit-serial approaches [5], [6], [7], [9]. Among these precision-scalable MAC (PSMAC) unit architectures, Fusion Unit (FU) [8] and its variants show excellent energy efficiency and throughput [12]. This brief focuses on FU-based PSMAC architectures unless otherwise specified.

A FU comprises 16 basic units called Bit Bricks (BBs) each of which can perform a $2b \times 2b$ multiplication. When configured into the high-precision mode, the partial products generated by BBs are shifted and added together to recover the final product. A FU usually supports at most an $8b \times 8b$ multiplication in one cycle and so does the PSMAC unit proposed in this brief. Obviously, it can achieve higher throughput in the low-precision modes than a traditional $8b \times 8b$ multiplier at the expense of area overhead. Specifically, the introduced shift-add (S&A) logic occupies more than a half of the overall area cost. By means of reducing the number of supported modes, a new FU architecture is proposed in [10] to lower the hardware complexity. However, the improvement is still limited.

The goal of this brief is to further reduce the complexity of the existing FU-based PSMAC architectures. The architecture of BB is first optimized through eliminating some redundant logic. We next devise a top-level architecture for PSMAC unit by employing BBs in a recursive manner. Then two FU-based PSMAC architectures are presented for two quantization schemes: one for general purpose and the other quantizes activations into nonnegative values. Furthermore, an insight into the decomposed multiplication is given and the bitwidths of the proposed architectures are accordingly reduced. These proposed optimization techniques are demonstrated to be effective by our experiments. It is also shown that our proposed architectures can significantly reduce the overall area cost and power consumption.

II. Fusion Unit and Related Works

Since the bitwidth varies significantly across DNNs and may even be individually adjusted for each layer within a DNN, the

Manuscript received 19 November 2022; accepted 16 December 2022. Date of publication 22 December 2022; date of current version 29 March 2023. This work was supported in part by the National Key Research and Development Program of China under Grant 2020YFB2205500, and in part by the National Natural Science Foundation of China under Grant 62074097. This brief was recommended by Associate Editor M. Huang. *(Corresponding authors: Ningyi Xu; Guanghui He.)*

The authors are with the School of Electronic Information and Electrical Engineering, Shanghai Jiao Tong University, Shanghai 200240, China (e-mail: xuningyi@sjtu.edu.cn; guanghui.he@sjtu.edu.cn).

Color versions of one or more figures in this article are available at https://doi.org/10.1109/TCSII.2022.3231418.

Digital Object Identifier 10.1109/TCSII.2022.3231418

1549-7747 © 2022 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: VIT University. Downloaded on February 07,2026 at 10:45:11 UTC from IEEE Xplore. Restrictions apply.

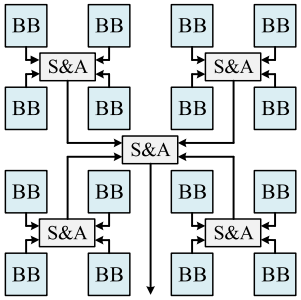


Fig. 1. A FU consists of 16 BBs and some S&A units.

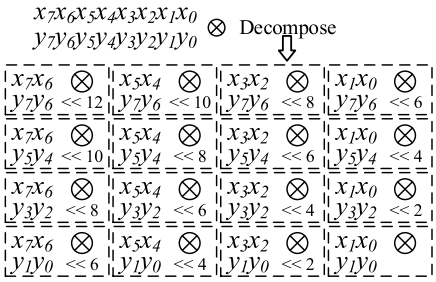


Fig. 2. A decomposed $8b \times 8b$ multiplication in which each sub-multiplication is handled by a BB.

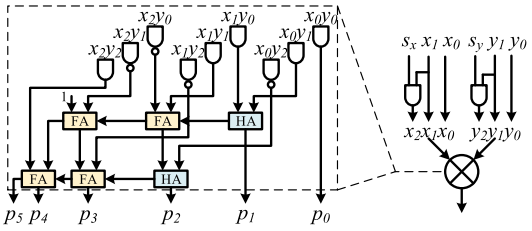


Fig. 3. The architecture of BB which is based on Baugh-Wooley multiplication between two $3b$ signed numbers.

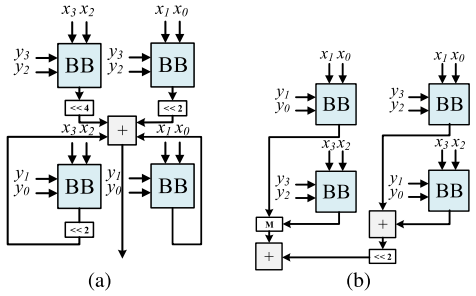


Fig. 4. Computation illustration for $4b \times 4b$ decomposed multiplications executed by a) the original FU [8], and b) the MFU [10].

benefits brought by fixed-bitwidth accelerators are limited. To take full advantage of the bitwidth variety, a bit-level dynamically composable accelerator architecture which is referred to as *Bit Fusion* is presented in [8]. It can achieve excellent performance in terms of throughput and energy efficiency.

The PSMAC unit employed in Bit Fusion is called FU which is composed of 16 BBs and some S&A units, as depicted in Fig. 1. The supported bitwidths for the input activations and weights include $2b$, $4b$ and $8b$. In the configuration mode of $2b \times 2b$, each BB computes a product of a $2b$ weight and a $2b$ activation. In a configuration mode for higher precision, each BB computes a sub-product that will be shifted and added with others to recover the final result(s). For example, Fig. 2 shows how an $8b \times 8b$ multiplication is decomposed such that each sub-multiplication is handled by a BB. Except in the $8b \times 8b$ mode, a FU can perform multiple multiplications between activations and weights and add their products together, i.e., obtaining an inner product of an activation vector and a weight vector. It provides Bit Fusion higher throughput than the conventional accelerators that adopt fixed-bitwidth MAC units. As input bitwidth decreases, more multiplications can be simultaneously performed by a FU.

For different modes or/and different quantization schemes, the inputs of BB may be signed or unsigned. Thus each of them has to be assigned with one extended sign bit to support both signed and unsigned numbers. To this end, one BB is set to perform a multiplication between two $3b$ signed numbers [8]. Its architecture based on Baugh-Wooley multiplication is given in Fig. 3 where "HA" and "FA" represent half adder and full adder, respectively. s_x and s_y are the flag signals to indicate whether the inputs are signed or unsigned numbers (1 for signed and 0 for unsigned). In addition to the 16 BBs, S&A units are adopted in FU, which take more than a half of the overall area cost.

It is pointed out in [10] that the accuracy gap incurred by different precisions is acceptable in some cases. Consequently, the granularity of the original FU is not necessary. It suggests to only support three configuration modes: $8b \times 8b$, $4b \times 4b$ and $2b \times 2b$. To save the S&A logic, [10] devises a merge-based FU (MFU). For the sake of brevity, we henceforth use $x_{j:i}$ to denote the binary representation of $(x_j x_{j-1}, \ldots x_{i+1} x_i)_2$. Take a $4b \times 4b$ decomposed multiplication $x_{3:0} \times y_{3:0}$ as an example. Four sub-products will be computed, shifted and added together. Note that $x_{1:0}$ and $y_{1:0}$ are both unsigned numbers and their sub-product is of at most four bits. Therefore [10] directly merges the sub-products of $x_{3:2} \times y_{3:2}$ and $x_{1:0} \times y_{1:0}$. Meanwhile, sub-products of $x_{3:2} \times y_{1:0}$ and $x_{1:0} \times y_{3:2}$ are added together and then shifted. Finally, the two intermediate results are added up to recover the product. The process can be formulated as $x_{3:0} \times y_{3:0} = merge\{x_{3:2} \times y_{3:2}, x_{1:0} \times y_{1:0}\} + ((x_{3:2} \times y_{1:0} + x_{1:0} \times y_{3:2}) << 2)$. The computation illustration for the $4b \times 4b$ multiplications executed by the original FU and MFU are depicted in Fig. 4. Clearly, Fig 4. (b) needs fewer shifters and two-input adders.

Fig. 5 illustrates the MFU architecture devised in [10]. It can be configured into three modes: $2b \times 2b$, $4b \times 4b$ and $8b \times 8b$. The configuration is handled by the MUXs. Fig. 5 also shows the data-path when MFU is configured into the $8b \times 8b$ mode. In this mode, the multiplication is decomposed into four parts: $x_{7:0} \times y_{7:0} = x_{1:0} \times y_{7:0} + ((x_{3:2} \times y_{7:0}) << 2) + ((x_{5:4} \times y_{7:0}) << 4) + ((x_{7:6} \times y_{7:0}) << 6)$. The four parts are respectively handled by the four columns of BBs. Analogous to the $4b \times 4b$ case, there exist a few merging operations. Such architecture saves the S&A logic and hence the overall area cost. However, as we will see in Section IV, the improvement brought by MFU is still limited.

Authorized licensed use limited to: VIT University. Downloaded on February 07,2026 at 10:45:11 UTC from IEEE Xplore. Restrictions apply.

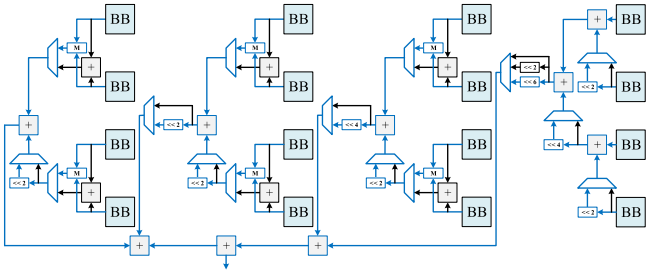


Fig. 5. Architecture of MFU [10] (data-path in $8b \times 8b$ mode is marked in blue).

TABLE I
PRODUCTS AND THEIR TWO'S COMPLEMENT REPRESENTATIONS FOR ALL THE POSSIBLE MULTIPLICATION CASES ENCOUNTERED BY A BB

Product	-6	-4	-3	-2	-1	0
$p_4p_3p_2p_1p_0$	11010	11100	11101	11110	11111	00000
Product	1	2	3	4	6	9
$p_4p_3p_2p_1p_0$	00001	00010	00011	00100	00110	01001

III. PROPOSED PRECISION-SCALABLE MULTIPLY-ACCUMULATE UNIT ARCHITECTURES

This section provides a detailed discussion on our proposed PSMAC architectures. We first propose one architecture for general purpose, i.e., supporting all the uniform quantization schemes. Considering the prevalent quantization scheme in which activations are quantized to nonnegative numbers, we propose another architecture that can achieve lower complexity. Furthermore, bitwidth optimization is applied to the proposed architectures.

A. Proposed PSMAC Unit for General Purpose

Some recently reported works [14], [15], [17] show that a quantization scheme only supporting $8b \times 8b$ or/and $4b \times 4b$ computation case(s) can achieve good tradeoff on model size and accuracy. Besides, ternary neural networks (TNNs) [18], [19], [20] whose weights and activations are restricted to $\{-1, 0, +1\}$ are good alternatives in low-accuracy application scenarios. As a result, we choose to follow MFU [10] which supports three configuration modes: $2b \times 2b$, $4b \times 4b$ and $8b \times 8b$.

Now we first optimize the architecture of BB. In effect, five bits are sufficient to represent the product of two $3b$ signed numbers. In other words, p_5 in Fig. 3 can be removed. Recall that each input of BB is extended by one sign bit. Thus the possible values of $x_{2:0}$ and $y_{2:0}$ are actually the same to that of the unextended 2-bit numbers which are restricted to $\{0, \pm1, \pm2, +3\}$. All the possible products are listed in Table I. In most cases, p_4 only depends on the sign bits of the inputs, i.e., $p_4 = x_2 \oplus y_2$. One exception is that p_4 will be zero when one input is zero and the other is a negative number. Consequently, we have $p_4 = (x_2 \oplus y_2)(x_1 + x_0)(y_1 + y_0)$ where "+" represents OR operations. Since p_5 has been already removed, the full adder outputting p_4 and the AND gate corresponding to x_2 and y_2 can be replaced by the circuits corresponding to the new computing formula for p_4. Moreover, the two full adders vertically corresponding to p_3 can also be simplified since their carry bits are not required anymore. As the simplified BB (SBB) and BB perform the same computation, any architectures employing BB (e.g., FU and MFU) can be accordingly modified by replacing BB with SBB.

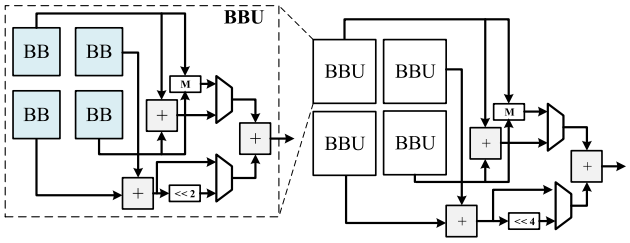


Fig. 6. Architecture of the proposed PSMAC which is called RFU.

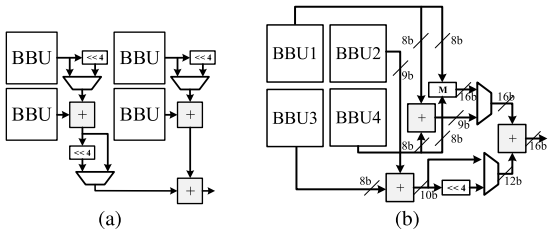


Fig. 7. (a) Architecture of the extended RFU which can additionally support the $8b \times 4b$ mode. (b) Bitwidths details of the circuits outside BBUs but in nRFU.

We adopt four BBs to construct a BB unit (BBU) that can be configured into $2b \times 2b$ or $4b \times 4b$ mode. The architecture of BBU is given in Fig. 6. When it is configured into the $4b \times 4b$ mode, the computation process is the same to Fig. 4 (b) except the input data arrangement. In the $2b \times 2b$ mode, the outputs of the left-hand two adders will pass through the MUXs to compute the inner product of weights and activations. To execute an $8b \times 8b$ multiplication, one can divide each operand into two 4-bit numbers and then compute four sub-products: $P_1 = x_{7:4} \times y_{7:4}$, $P_2 = x_{7:4} \times y_{3:0}$, $P_3 = x_{3:0} \times y_{7:4}$, $P_4 = x_{3:0} \times y_{3:0}$. Similar to Fig. 4 (b), the final product can be recovered by $x_{7:0} \times y_{7:0} = merge\{P_1, P_4\} + ((P_2 + P_3) << 4)$. Fig. 6 also depicts the architecture of the proposed PSMAC which consists of four BBUs. Clearly, it is constructed by recursively employing BB to support both $4b \times 4b$ and $8b \times 8b$ configuration modes. The top-level architecture of the proposed PSMAC is almost the same to that of BBU, except the shifters and circuit bitwidths. We refer to the proposed architecture as recursive FU (RFU). In the $8b \times 8b$ mode, the input data is arranged as shown in Fig. 2.

In some mixed-precision quantization schemes [3], [14], there exist a few layers whose activations and weights are quantized using eight and four bits, respectively. MFU is not easy to be adjusted to additionally support the configuration mode $8b \times 4b$. Instead, our proposed RFU can be easily extended to additionally support the $8b \times 4b$ configuration mode at the expense of moderate overhead. The extended RFU is shown in Fig. 7 (a). When configured into the $8b \times 4b$ mode, each column of BBUs are in charge of one $8b \times 4b$ multiplication. In the $8b \times 8b$ mode, two columns of BBs compute two sub-products P_{left} and P_{right}. The final result is recovered as $(P_{left} << 4) + P_{right}$.

Authorized licensed use limited to: VIT University. Downloaded on February 07,2026 at 10:45:11 UTC from IEEE Xplore. Restrictions apply.

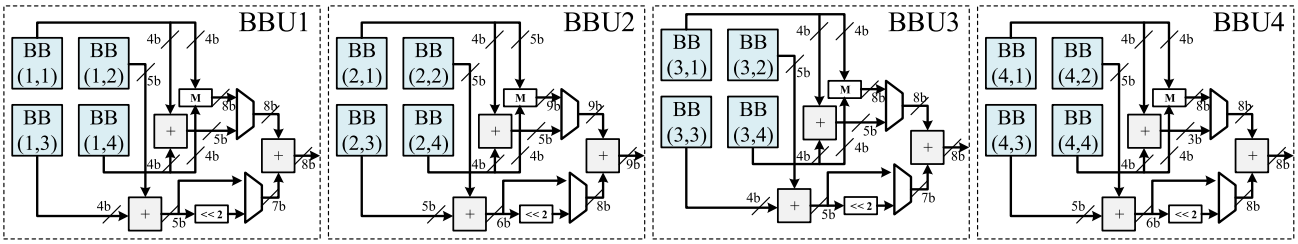


Fig. 8. Bitwidth details of the BBUs in nRFU.

B. Adaptation for Nonnegative-Activation Quantization and Bitwidth Optimization

Some quantization schemes such as the two proposed by Google [13] and Qualcomm [21] quantize the activations into the interval [0, 255] using eight bits, i.e., uint8. In effect, activations are usually quantized into nonnegative numbers since ReLU has been widely adopted as the activation functions of DNNs. Due to the nonnegative-quantized activations, the architecture of SBB can be further simplified through setting s_y (see Fig. 3) to zero. On the other hand, we have $p_4 = x_2(y_1+y_0)$. While activations are quantized into unsigned numbers, weights are always quantized into signed numbers in these quantization schemes. Consequently, we can set $s_x = 1$ to save another AND gate for each SBB. The further simplified BB is denoted as nSBB where "n" is the abbreviation of "nonnegative". RFU employing nSBBs is referred to as nRFU.

Conventional TNNs are not supported by nRFU anymore since nSBB only works for the nonnegative-quantized activations. Fortunately, some nonnegative-activation quantization schemes [16], [17] yield acceptable accuracy even aggressively quantizing both the weights and activations into two bits. At this point, nRFU can be configured into the $2b \times 2b$ mode to support the 2-bit quantization. Note that RFU is also effective for the nonnegative-activation quantization schemes. However, it is not the most efficient choice in this situation.

Now we will give an insight into the decomposed multiplication and accordingly reduce the bitwidths of RFU and nRFU. Considering all the possible values of each signal, bitwidths may be reduced from the naive implementation. For RFU, the benefits brought by such bitwidth optimization are not significant. The main reason is that only a few signals successfully reduce their bitwidths due to the high generality of RFU. For example, all BBs have to output five bits to support all the possible product values as listed in Table I. As to nRFU, bitwidth optimization can lead to more improvement. Bitwidth details of the circuits outside BBUs but in nRFU are shown in Fig. 7 (b). Meanwhile, Fig. 8 provides bitwidth details of the BBUs. As listed in Table I, four bits are sufficient for the output of BB except the case of $3 \times 3 = 9$. In the $2b \times 2b$ mode, the 3×3 case will never happen since the quantized weights are always 2-bit signed numbers. In the $4b \times 4b$ mode, only the lower-right BB of each BBU is likely to perform the 3×3 multiplication. Nevertheless, its output is treated as the least significant part in the merging operation. Hence the sign bit can be removed so that four bits are also sufficient for the lower-right BB. Five-bit outputs are required by some BBs only in the $8b \times 8b$ mode. For these BBs with only four output bits, their architectures can be further simplified. By means of optimizing the bitwidth, the complexity of RFU and nRFU is further reduced, which is not considered neither in [8] nor in [10]. Similarly, the bitwidths of FU and MFU can also be optimized, which will not be discussed in detail due to the space limitation.

TABLE II
SYNTHESIS RESULTS FOR DIFFERENT BBS

BB Design	Original [8]	SBB	nSBB	4b-SBB	4b-nSBB
Area	31.93	28.93	16.58	23.11	15.70
Power	3.89	3.32	1.77	2.55	1.71

* Units of Area and Power are um^2 and 10^{-3} mW, respectively.

TABLE III
AREA (um^2) BREAKDOWN OF DIFFERENT FU-BASED PSMAC ARCHITECTURES

Type of BB	Original [8]	SBB	nSBB	SBB BO	nSBB BO
FU [8]					
Overall	2440.49	2263.92	1830.86	1699.08	1154.54
BB	636.80	595.88	200.39	482.98	194.04
S&A	1803.69	1668.04	1630.47	1216.10	960.50
MFU [10]					
Overall	2299.90	2027.37	1525.86	1849.73	1161.42
BB	597.29	564.66	227.91	561.66	158.59
S&A	1702.61	1462.71	1297.95	1288.07	1002.83
Proposed RFU or nRFU					
Overall	1485.99	1324.76	993.84	1283.84	893.47
BB	477.16	467.28	167.93	471.87	169.70
S&A	1008.83	857.48	825.91	811.97	723.77

* "BO" is the abbreviation of "Bitwidth Optimization".

IV. EXPERIMENTAL RESULTS AND COMPARISONS

In this section, experimental results are provided to demonstrate the superiority of the proposed PSMAC architectures. All the synthesis results are obtained using the TSMC 40-nm CMOS technology and with voltage of 0.99 V. The clock period is set to 1.5 ns for all designs.

Table II provides the synthesis results for different BB designs. While SBB can save 9.40% area cost and 14.65% power consumption, the reduction ratios brought by nSBB are up to 48.07% and 54.50%. As discussed in Section III-B, some BBs can be further optimized since they only need to output four bits. For example, nSBB with four output bits saves up to 50.83% area cost and 56.04% power consumption of the original one.

Table III gives the area breakdown for the three kinds of FU-based PSMAC architectures. As S&A logic dominates the area cost in [8] and [10], the simplification on BB has a small impact on the overall architecture. Fortunately, the proposed

Authorized licensed use limited to: VIT University. Downloaded on February 07,2026 at 10:45:11 UTC from IEEE Xplore. Restrictions apply.

TABLE IV
COMPARISONS ON THE PROPOSED AND EXISTING FU-BASED PSMAC ARCHITECTURES

Architecture	Area (um^2)	Power (mW)	Quan. Scheme	Supported Modes
FU [8]	2440.49	0.61	General	$2b/4b/8b \times 2b/4b/8b$
MFU [10]	2299.90	0.55	General	$2b \times 2b$, $4b \times 4b$, $8b \times 8b$
RFU	**1283.84**	0.30	General	$2b \times 2b$, $4b \times 4b$, $8b \times 8b$
nRFU	**893.47**	0.21	NAQ	$2b \times 2b$, $4b \times 4b$, $8b \times 8b$
Extended RFU	**1428.13**	0.34	General	$2b \times 2b$, $4b \times 4b$, $8b \times 4b$, $8b \times 8b$
Extended nRFU	**943.74**	0.23	NAQ	$2b \times 2b$, $4b \times 4b$, $8b \times 4b$, $8b \times 8b$

* NAQ represents Nonnegative-Activation Quantization.

recursive architecture surpasses the previous two designs and successfully decreases the proportion of S&A logic regardless of the type of employed BBs. Assume the same BB is employed, e.g., the original BB. Our proposed recursive architecture can still reduce the area cost up to 39.11% and 35.39% when compared with [8] and [10].

Table III also lists the implementation results after applying the bitwidth optimization. Bitwidth optimization has a direct impact on S&A logic and some BBs can also be simplified by decreasing their outputs to four bits. For all the cases, bitwidth optimization brings 3.09%∼36.94% area reduction.

A direct comparison on our proposed PSMAC architectures and the existing two other FU-based designs is given in Table IV. Targeted at general-purpose quantization schemes, our proposed RFU can save 44.18% area cost and 45.45% power consumption when compared with MFU. Regarding the original FU, the two reduction ratios brought by RFU will be 47.39% and 50.82%. Moreover, as aforementioned, the extended RFU additionally supports the configuration mode $8b \times 4b$ while introduces only an overhead of 11.24%. Hence, the extended RFU outperforms MFU in terms of area cost, power consumption and configuration flexibility. Note that nonnegative-quantized activations are not considered by FU and MFU. On the contrary, this brief develops an optimized PSMAC architecture dedicated to it. When nonnegative-activation quantization schemes [13], [21] are adopted, nRFU is more efficient than those proposed in [8] and [10].

The PSMAC architectures are proposed without imposing any additional constraint on top-level accelerator architecture. Therefore, they are practical for all the FU-based precision-scalable accelerators. As reported by [10], the processing element (PE) array of its design consumes more than 47% power. Recall that the proposed RFU can reduce the power of MFU by 45.45%. Directly replacing MFU by RFU is expected to save the overall power of the accelerator by around 20%. Another example is the accelerator presented in [11]. According to its reported results, the PE array occupies over 60% chip area. The benefits brought by our proposed PSMAC architectures will be more significant.

V. CONCLUSION

This brief proposes two low-complexity PSMAC architectures for two different quantization schemes. They are obtained through simplifying the basic unit BB, devising a new top-level architecture and applying bitwidth optimization. Experimental results demonstrate their superiority in terms of area cost and power consumption. A brief discussion is also given to show that the proposed PSMAC units can bring considerable benefits to some of the existing precision-scalable DNN accelerators.

REFERENCES

[1] V. Sze, Y.-H. Chen, T.-J. Yang, and J. S. Emer, "Efficient processing of deep neural networks: A tutorial and survey," *Proc. IEEE*, vol. 105, no. 12, pp. 2295–2329, Dec. 2017.

[2] Z. Dong, Z. Yao, A. Gholami, M. W. Mahoney, and K. Keutzer, "HAWQ: Hessian aware quantization of neural networks with mixed-precision," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2019, pp. 293–302.

[3] Y. Cai, Z. Yao, Z. Dong, A. Gholami, M. W. Mahoney, and K. Keutzer, "ZeroQ: A novel zero shot quantization framework," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2020, pp. 13166–13175.

[4] Y. Huang et al., "LSMQ: A layer-wise sensitivity-based mixed-precision quantization method for bit-flexible CNN accelerator," in *Proc. Int. SoC Des. Conf.*, 2021, pp. 256–257.

[5] P. Judd, J. Albericio, T. Hetherington, T. M. Aamodt, and A. Moshovos, "Stripes: Bit-serial deep neural network computing," in *Proc. 49th Annu. IEEE/ACM Int. Symp. Microarchitect.*, 2016, pp. 1–12.

[6] J. Albericio et al., "Bit-pragmatic deep neural network computing," in *Proc. 50th Annu. IEEE/ACM Int. Symp. Microarchitect.*, 2017, pp. 382–394.

[7] S. Sharify, A. D. Lascorz, K. Siu, P. Judd, and A. Moshovos, "Loom: Exploiting weight and activation precisions to accelerate convolutional neural networks," in *Proc. 55th Annu. Des. Autom. Conf.*, 2018, pp. 1–6.

[8] H. Sharma et al., "Bit fusion: Bit-level dynamically composable architecture for accelerating deep neural network," in *Proc. ACM/IEEE 45th Annu. Int. Symp. Comput. Archit.*, 2018, pp. 764–775.

[9] J. Lee, C. Kim, S. Kang, D. Shin, S. Kim, and H.-J. Yoo, "UNPU: An energy-efficient deep neural network accelerator with fully variable weight bit precision," *IEEE J. Solid-State Circuits*, vol. 54, no. 1, pp. 173–185, Jan. 2019.

[10] W. Liu, J. Lin, and Z. Wang, "A precision-scalable energy-efficient convolutional neural network accelerator," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 67, no. 10, pp. 3484–3497, Oct. 2020.

[11] S. Ryu et al., "BitBlade: Energy-efficient variable bit-precision hardware accelerator for quantized neural networks," *IEEE J. Solid-State Circuits*, vol. 57, no. 6, pp. 1924–1935, Jun. 2022.

[12] V. Camus, L. Mei, C. Enz, and M. Verhelst, "Review and benchmarking of precision-scalable multiply-accumulate unit architectures for embedded neural-network processing," *IEEE J. Emerg. Sel. Topics Circuits Syst.*, vol. 9, no. 4, pp. 697–711, Dec. 2019.

[13] B. Jacob et al., "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2018, pp. 2704–2713.

[14] Y. Li et al., "BRECQ: Pushing the limit of post-training quantization by block reconstruction," in *Proc. Int. Conf. Learn. Represent.*, 2021, pp. 1–16.

[15] Z. Yao et al., "HAWQ-V3: Dyadic neural network quantization," in *Proc. Int. Conf. Mach. Learn.*, 2021, pp. 11875–11886.

[16] S. Xu et al., "Generative low-bitwidth data free quantization," in *Proc. Eur. Conf. Comput. Vis.*, 2020, pp. 1–17.

[17] Y. Liu, W. Zhang, and J. Wang, "Zero-shot adversarial quantization," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2021, pp. 1512–1521.

[18] H. Alemdar, V. Leroy, A. Prost-Boucle, and F. Petrot, "Ternary neural networks for resource-efficient AI applications," in *Proc. Int. Joint Conf. Neural Netw.*, 2017, pp. 2547–2554.

[19] L. Deng, P. Jiao, J. Pei, Z. Wu, and G. Li, "GXNOR-Net: Training deep neural networks with ternary weights and activations without full-precision memory under a unified discretization framework," *Neural Netw.*, vol. 100, pp. 49–58, Apr. 2018.

[20] Y. Zhang et al., "When sorting network meets parallel bitstreams: A fault-tolerant parallel ternary neural network accelerator based on stochastic computing," in *Proc. Des. Autom. Test Eur. Conf. Exhibit.*, 2020, pp. 1287–1290.

[21] M. Nagel et al., "A white paper on neural network quantization," 2021, *arXiv:2106.08295*.

Authorized licensed use limited to: VIT University. Downloaded on February 07,2026 at 10:45:11 UTC from IEEE Xplore. Restrictions apply.